

CUSTOMER MILESTONE BRIEF

MS #20 - Trusted Extension Evidence Contract

Purpose, customer value, delivered capability, evidence, and claim boundaries

Status	Complete
Release	0.20.0
Date	2026-08-26
Audience	Customers, partners, executives, architects, delivery teams, and assurance reviewers

Executive summary

MS #20 - Trusted Extension Evidence Contract converts a specific part of the LIGHTYEAR modernization approach into a governed, reviewable capability. Defines how new language, platform, and adapter capabilities join the factory without bypassing its evidence and trust boundaries.

Why this matters to a customer

Defines how new language, platform, and adapter capabilities join the factory without bypassing its evidence and trust boundaries.

The practical result is a narrower, evidence-backed decision instead of a broad modernization claim. Commercial, architecture, delivery, security, and assurance teams can review the same capability, proof, and limits.

What the milestone delivers

- Added a versioned adapter evidence envelope that binds every claim to an exact graph identity and classifies evidence as live, recorded, simulated, or inferred.
- Added recursive credential redaction, bounded artifact metadata, optional HMAC signatures, and fail-closed validation for drift, tampering, missing graph entities, unsafe scope, and untrusted signing keys.
- Added a deterministic record/replay adapter that downgrades live captures to recorded evidence and can never promote simulated or inferred evidence.
- Added an adapter registry with implemented fixture, replay, and z/OSMF contracts plus explicit pre-access contracts for Db2 for z/OS catalog and CICS CMCI collectors.
- Added a hash-bound graph extension-fragment contract so new language packs do not silently mutate the verified base graph or invalidate downstream audit and runtime evidence.
- Added the first PL/I language pack proof for programs, procedures, includes, file access, embedded Db2 SQL, and a mixed-language call into the existing CardDemo COBOL graph.

- Added deterministic POSIX and PowerShell verification, JSON schemas, adversarial tests, and committed development receipts while keeping live-mainframe and production-readiness claims blocked.

How it differs from earlier work

MS #20 builds on MS #19 (DB2 and Multi-Target Data Modernization). The earlier milestone established its own bounded capability; this milestone adds trusted extension evidence contract while preserving the earlier evidence and its limits.

Earlier evidence is not automatically promoted into this milestone. A parser, simulator, local comparison, or generated artifact is not live platform equivalence or production readiness.

What a customer can do with it

A customer can use trusted extension evidence contract as a bounded decision and delivery capability, attached to approved sources, owners, policies, target architecture, acceptance criteria, and authorized evidence.

Unresolved semantic loss, policy decisions, unsupported behavior, and live-evidence requirements remain visible.

Evidence and acceptance posture

The repository capability is complete because its committed implementation and release record are present. Customer qualification remains claim-specific and evidence-class-specific.

Review exact source and artifact identities, distinguish development from authorized live evidence, inspect policy and loss classifications, confirm passed and blocked gates, and verify that later changes have not invalidated the evidence.

Boundaries and claims not made

- This milestone establishes only the bounded capabilities and evidence listed here; later capabilities are not retroactive.
- Production readiness and platform equivalence require the explicit evidence and policy gates applicable to the customer environment.

Source of record

- CHANGELOG.md release section(s): 0.20.0
- README.md product and operating guidance
- LIGHTYEAR-ROADMAP.md governing sequence and claim classifications
- docs/milestones/catalog.json metadata and docs/milestones/manifest.json artifact integrity